

Deploying an Angular Application Using Docker and Docker Compose

Development container with Angular CLI and production deployment with Nginx

Project	Project 2: Angular Docker Deployment
Submitted by	Kashyup Gaud
Course / Subject	DevOps Lab
Date	22 July 2026

Development Dockerfile

```
FROM node:22.22.3

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .
EXPOSE 4200
CMD ["npm", "start"]
```

The development image installs dependencies, copies the project, exposes Angular's development port, and executes the project's start script.

Production Dockerfile

```
# Build stage
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build -- --configuration production

# Production stage
FROM nginx:alpine
COPY --from=build /app/dist/angular-docker/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

The build tools remain in the temporary Node.js stage. Only Nginx and the compiled static files are retained in the final runtime image.

Development Compose File

```
services:
  angular-app:
    build:
      context: .
      dockerfile: Dockerfile.dev
    ports:
      - "4200:4200"
    environment:
      - NG_ALLOWED_HOSTS=localhost
    volumes:
      - ./app
      - /app/node_modules
```

Production Compose File

```
services:

angular-prod:

  build:
    context: .
    dockerfile: Dockerfile

  ports:
    - "80:80"
```

Implementation Evidence and Screenshot Analysis

The following figures document the configuration, execution, validation, and shutdown of the project. Each screenshot is paired with an explanation of the evidence it provides.

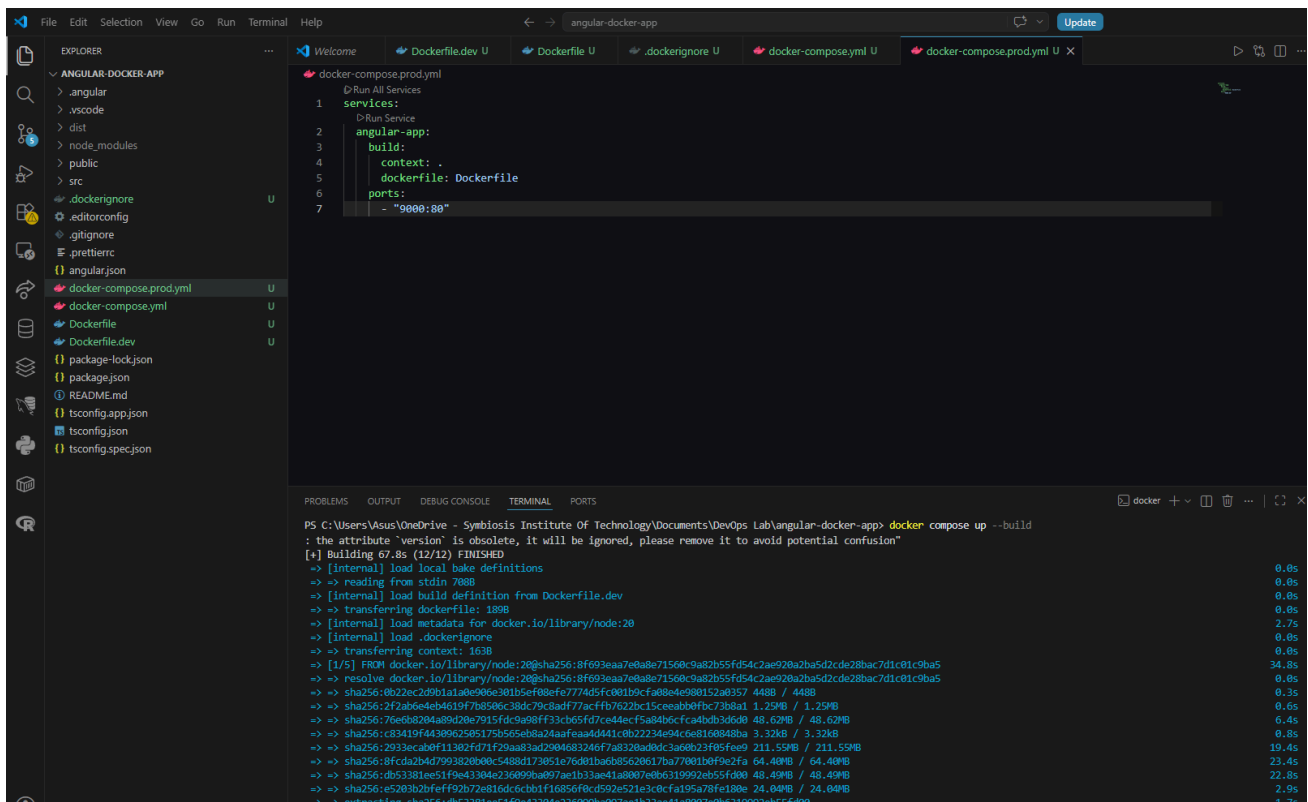
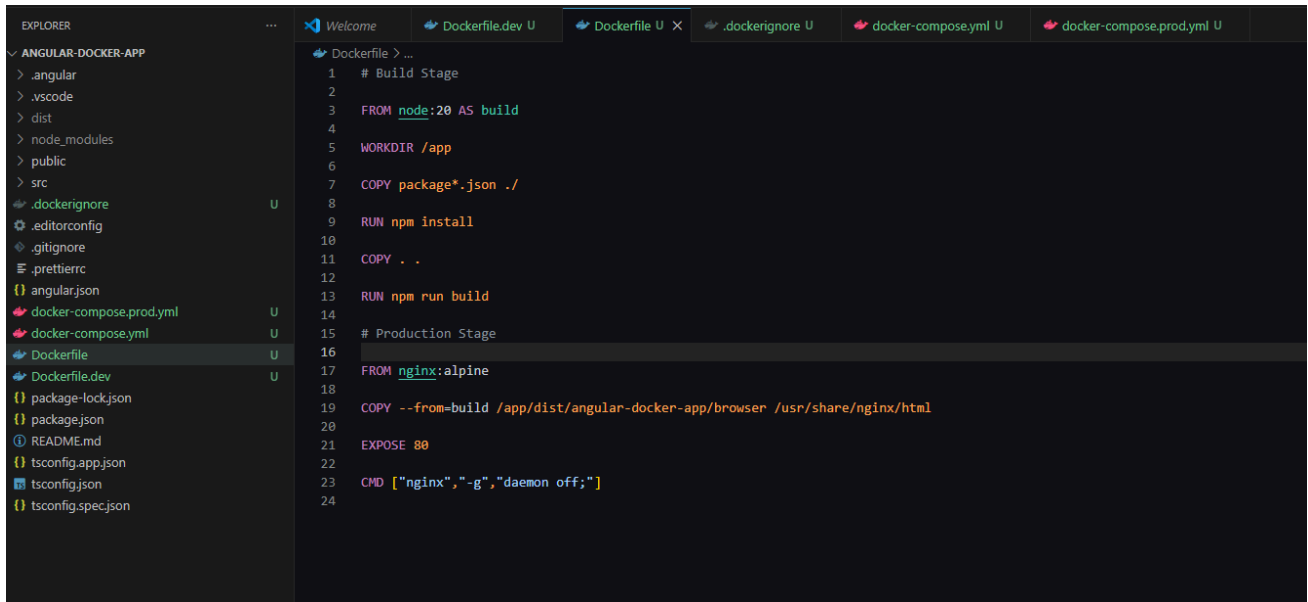


Figure 2. Angular project structure in VS Code

Explanation: The Explorer shows the Angular source folders and all required Docker configuration files: Dockerfile, Dockerfile.dev, docker-compose.yml, docker-compose.prod.yml, nginx.conf, package files, and TypeScript configuration files.

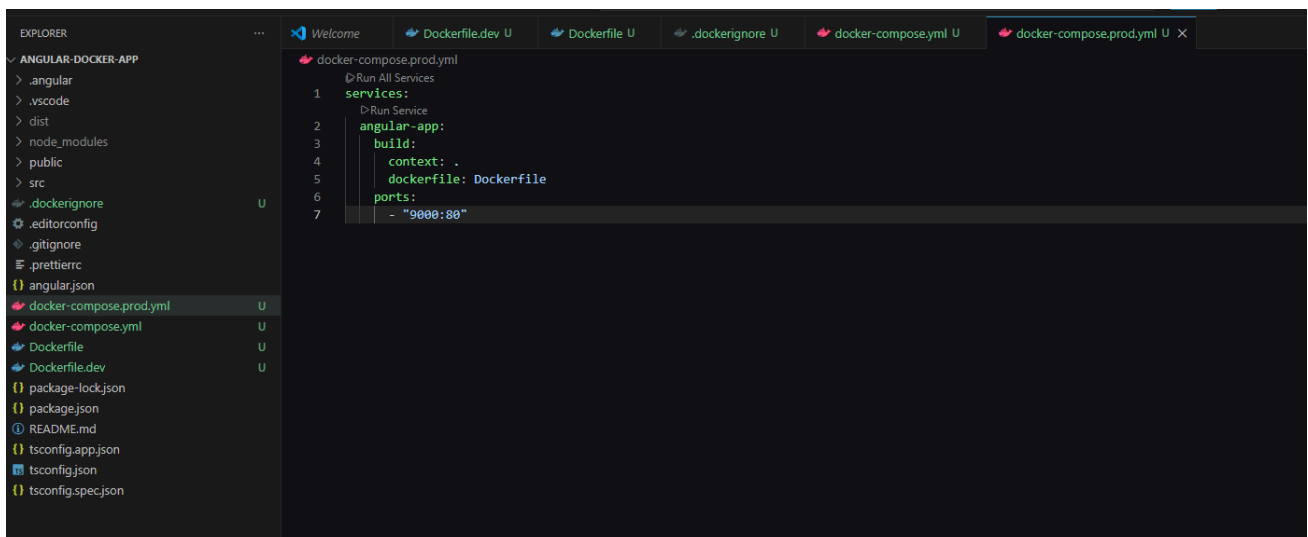


The screenshot shows the Visual Studio Code interface with the Explorer on the left and the Dockerfile editor on the right. The Explorer shows a project named 'ANGULAR-DOCKER-APP' with various files and folders. The Dockerfile editor shows the following content:

```
1 # Build Stage
2
3 FROM node:20 AS build
4
5 WORKDIR /app
6
7 COPY package*.json ./
8
9 RUN npm install
10
11 COPY . .
12
13 RUN npm run build
14
15 # Production Stage
16
17 FROM nginx:alpine
18
19 COPY --from=build /app/dist/angular-docker-app/browser /usr/share/nginx/html
20
21 EXPOSE 80
22
23 CMD ["nginx", "-g", "daemon off;"]
24
```

Figure 3. Development Dockerfile

Explanation: The development Dockerfile uses Node.js, sets /app as the working directory, installs npm dependencies, copies the application, exposes port 4200, and starts the Angular application.



The screenshot shows the Visual Studio Code interface with the Explorer on the left and the docker-compose.prod.yml editor on the right. The Explorer shows the same project as Figure 3. The docker-compose.prod.yml editor shows the following content:

```
1 services:
2   # Run Service
3   angular-app:
4     build:
5       context: .
6       dockerfile: Dockerfile
7     ports:
8       - "9000:80"
```

Figure 4. Multi-stage production Dockerfile

Explanation: The first stage builds the Angular application with Node.js. The second stage uses Nginx Alpine, copies the browser build into the Nginx document root, exposes port 80, and keeps Nginx in the foreground.

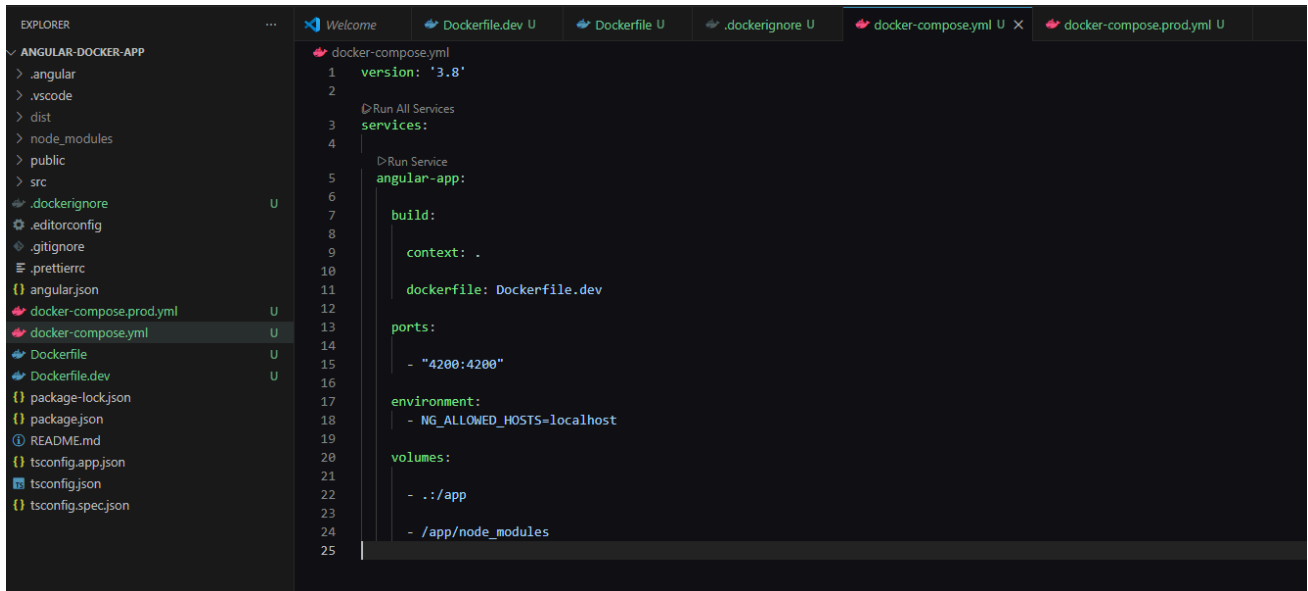


Figure 5. Development Docker Compose configuration

Explanation: The development service builds from Dockerfile.dev, maps host port 4200 to container port 4200, passes the allowed-host setting, mounts the project directory, and protects container-installed node_modules with an anonymous volume.

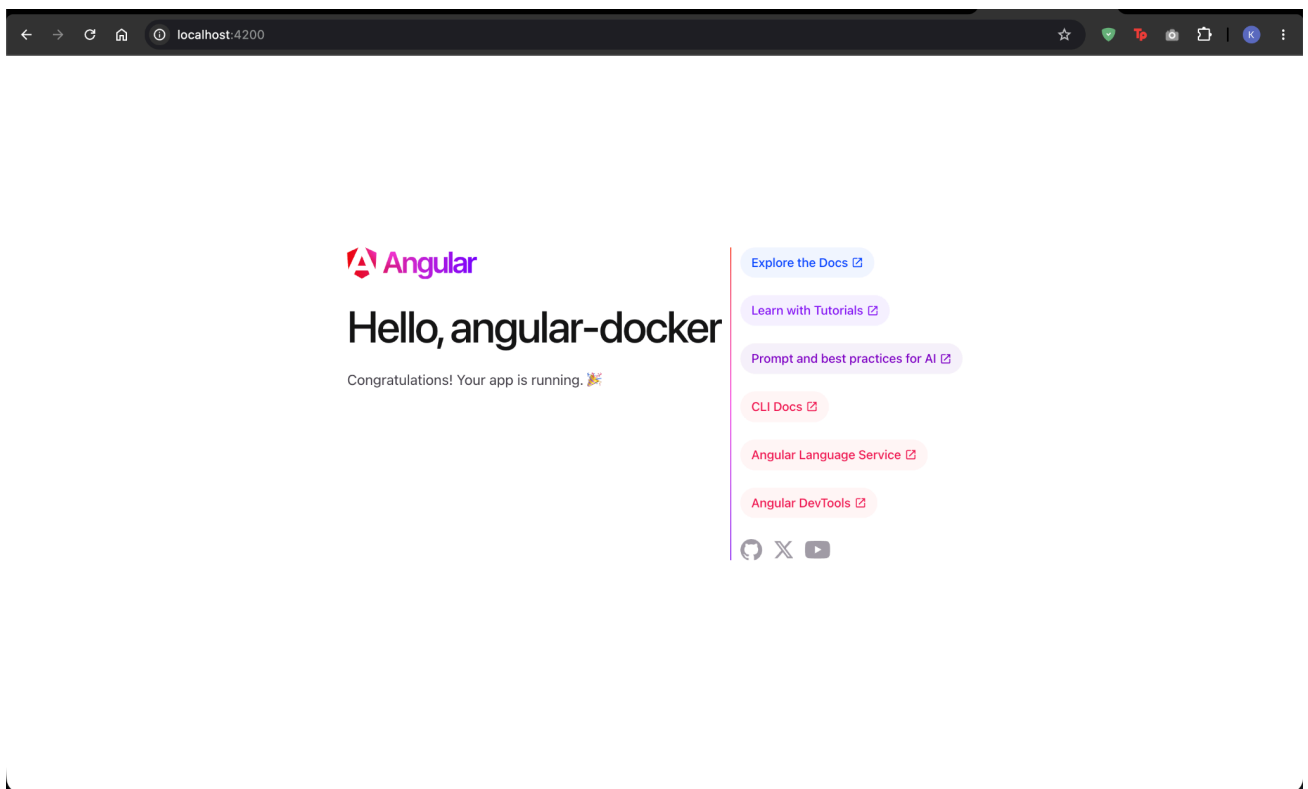


Figure 7. Angular development application on port 4200

Explanation: The browser successfully loads the Angular starter page from localhost:4200, proving that the development container is running and the port mapping is correct.

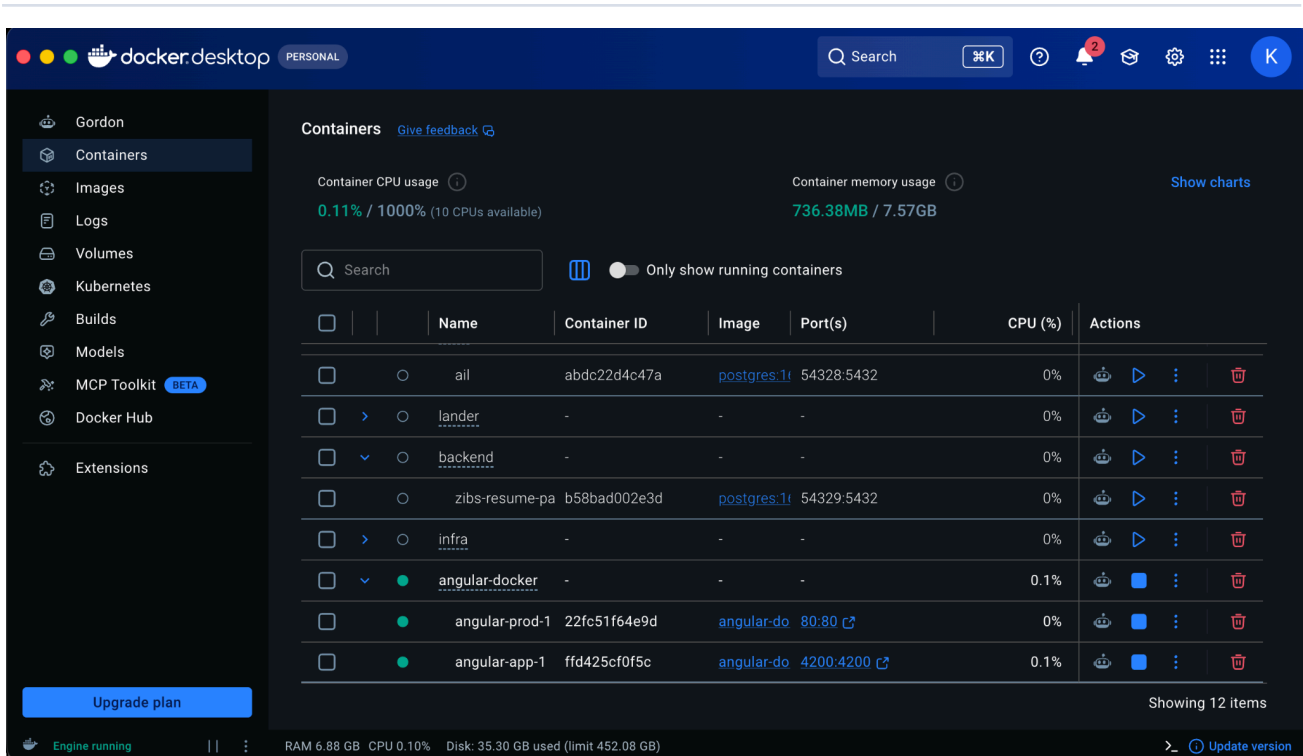


Figure 8. Running Angular containers in Docker Desktop

Explanation: Docker Desktop shows the angular-docker Compose project and its running development and production containers. The published ports 4200:4200 and 80:80 are visible.

```
kisna@MacBook-Air-2 angular-docker % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
fbd8c6b3c2c8   angular-app                          "/docker-entrypoint..." 45 seconds ago Up 45 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
22fc51f64e9d   angular-docker-angular-prod         "/docker-entrypoint..." 3 minutes ago  Up 3 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp
cp             angular-docker-angular-prod-1
kisna@MacBook-Air-2 angular-docker %
```

Figure 9. Running containers verified with docker ps

Explanation: The terminal lists the running Angular images, container status, and host-to-container port mappings. This confirms that Docker has published the required ports.

```
kisna@MacBook-Air-2: angular-docker % docker compose -f docker-compose.prod.yml up --build
WARN[0000] /Users/kisna/College/DevOps/Projects/DevOps-Lab/Project-2/angular-docker/docker-compose.prod.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 12.7s (17/17) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 634b 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 395B 0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 0.0s
=> [internal] load metadata for docker.io/library/node:22-alpine 0.9s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [build 1/6] FROM docker.io/library/node:22-alpine@sha256:16e22a558f7863286a1701448c45f 0.0s
=> => resolve docker.io/library/node:22-alpine@sha256:16e22a558f7863286a1701448c45f7912c6 0.6s
=> [internal] load build context 0.0s
=> => transferring context: 1.85MB 0.6s
=> [stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:20316569d8f81a160065d7d2a5eff 0.1s
=> => resolve docker.io/library/nginx:alpine@sha256:20316569d8f81a160065d7d2a5eff7c9a97d7 0.0s
=> CACHED [build 2/6] WORKDIR /app 0.0s
=> [build 3/6] COPY package*.json ./ 0.0s
=> [build 4/6] RUN npm install 4.2s
=> [build 5/6] COPY . . 1.8s
=> [build 6/6] RUN npm run build -- --configuration production 4.8s
=> [stage-1 2/3] COPY --from=build /app/dist/angular-docker/browser /usr/share/nginx/html 0.0s
=> [stage-1 3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf 0.0s
=> => exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:15f6816c1ca720b68860433a9f9052c941f53d4d07e919c3da39378e8d 0.0s
=> => exporting config sha256:9a4030d59ee21b447bdc3b01c7813e3c224b4d449d206cece927d889cae8 0.0s
=> => exporting attestation manifest sha256:d3aded1be7e304cd8c903b1f8bb97709322835ab60af47 0.0s
=> => exporting manifest list sha256:eb22183658debc36f0a5b64752229494f1cbe6ead3e5857e2c 0.1s
=> => naming to docker.io/library/angular-docker-angular-prod:latest 0.0s
=> => unpacking to docker.io/library/angular-docker-angular-prod:latest 0.0s
=> => resolving provenance for metadata file 0.0s
WARN[0012] Found orphan containers ([angular-docker-angular-app-1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
[+] up 2/2
✔ Image angular-docker-angular-prod Built 12.8s
✔ Container angular-docker-angular-prod-1 Created 0.0s
Attaching to angular-prod-1
angular-prod-1 /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
angular-prod-1 /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
angular-prod-1 /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
angular-prod-1 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
angular-prod-1 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
angular-prod-1 /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
angular-prod-1 /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
angular-prod-1 /docker-entrypoint.sh: Configuration complete; ready for start up
angular-prod-1 2026/07/22 07:26:03 [notice] #1: using the "epoll" event method
angular-prod-1 2026/07/22 07:26:03 [notice] #1: nginx/1.31.2
angular-prod-1 2026/07/22 07:26:03 [notice] #1: built by gcc 15.2.0 (Alpine 15.2.0)
angular-prod-1 2026/07/22 07:26:03 [notice] #1: OS: Linux 6.12.76-linuskitt
angular-prod-1 2026/07/22 07:26:03 [notice] #1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker processes
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 29
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 30
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 31
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 32
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 33
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 34
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 35
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 36
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 37
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 38
angular-prod-1 192.168.65.1 - - [22/Jul/2026:07:26:35 +0000] "GET / HTTP/1.1" 200 23141 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" "-"
angular-prod-1 192.168.65.1 - - [22/Jul/2026:07:26:35 +0000] "GET /main-QKQp7UUE.js HTTP/1.1" 200 253288 "http://localhost/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" "-"
angular-prod-1 192.168.65.1 - - [22/Jul/2026:07:26:35 +0000] "GET /styles-N3GcZVBPS.css HTTP/1.1" 200 4544 "http://localhost/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" "-"
angular-prod-1 192.168.65.1 - - [22/Jul/2026:07:26:35 +0000] "GET /favicon.ico HTTP/1.1" 200 15886 "http://localhost/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" "-"
[+] View in Docker Desktop  View Config  View Logs  Enable Watch  Detach
```

Figure 10. Production image build and Nginx startup

Explanation: The production Compose command completes the Node.js build stage, copies the Angular browser output into Nginx, creates the container, starts Nginx workers, and records successful HTTP 200 responses.

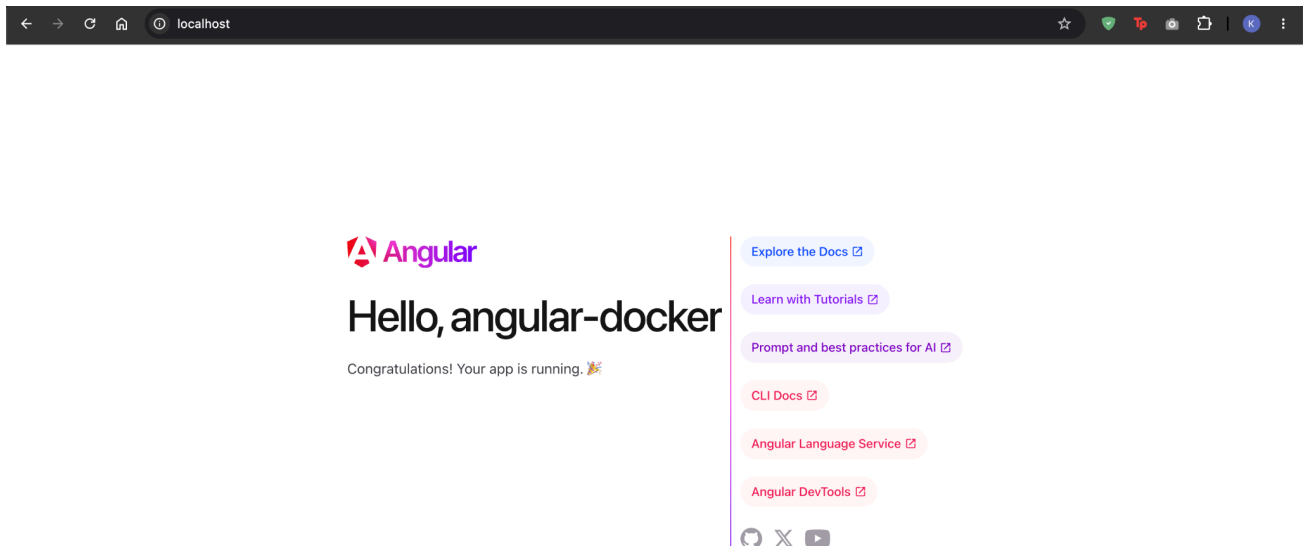


Figure 11. Production Angular application served by Nginx

Explanation: The Angular application loads at http://localhost without an explicit port because the production service is published on the standard HTTP port 80.

```
C:\Users\Asus\OneDrive - Symbiosis Institute Of Technology\Documents\DevOps Lab\angular-docker-app>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ail-backend:latest	d04d1fb690de	3.21GB	875MB	U
ail-frontend:latest	a196962fcf1a	857MB	196MB	U
ail-kafka-consumer:latest	60fc2e16b49c	411MB	94.4MB	U
ail-kafka-producer:latest	1f17dbfe7326	411MB	94.4MB	U
angular-docker-angular-app:latest	eaaffded207eb	1.25GB	275MB	U
angular-docker-app-angular-app:latest	67e34474b4af	93.1MB	26.2MB	U
ankane/pgvector:latest	956744bd14e9	628MB	160MB	U
confluentinc/cp-kafka:7.5.0	fb6b6fa11b25	1.33GB	439MB	U
confluentinc/cp-kafka:latest	68cf7eb0d994	865MB	299MB	U
confluentinc/cp-zookeeper:7.5.0	02f6c042bb9a	1.33GB	439MB	U
confluentinc/cp-zookeeper:latest	7610a50b13e7	1.75GB	593MB	U
hello-world:latest	d4aaab6242e0	25.9kB	9.52kB	U
my-flask-app:latest	f74cf038b7e1	200MB	48.8MB	U
pbl-2-neuro-adaptive-ai-assistant-backend:latest	7fd27bc761b2	1.1GB	245MB	U
pbl-2-neuro-adaptive-ai-assistant-frontend:latest	de1b0bc6e2d7	1.5GB	401MB	U
postgres:16	2586e2a95d1c	641MB	166MB	U
subtrack-backend:latest	ec0b2ad6a724	1.13GB	272MB	U
subtrack-frontend:latest	5f2af9eee49e	93.8MB	26.3MB	U

Figure 12. Docker images available on the host

Explanation: The docker images command lists the created Angular development and production images alongside the base Node.js and Nginx images used by the project.

```
kisna@MacBook-Air-2 angular-docker % docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
angular-docker-angular-app-1	angular-docker-angular-app	"docker-entrypoint.s..."	angular-app	10 minutes ago	Up 10 minutes	0.0.0.0:4200->4200/tcp, [::]:4200->4200/tcp
angular-docker-angular-prod-1	angular-docker-angular-prod	"/docker-entrypoint..."	angular-prod	15 minutes ago	Up 15 minutes	0.0.0.0:80->80/tcp, [::]:80->80/tcp

Figure 13. Docker Compose service status

Explanation: docker compose ps displays both Compose services, their images, running state, creation time, and published ports.

```
kisna@MacBook-Air-2 angular-docker % docker compose down
```

```
[+] down 2/2
✓ Container angular-docker-angular-app-1 Removed 0.7s
! Network angular-docker_default Resource is still in use 0.6s
kisna@MacBook-Air-2 angular-docker % docker compose -f docker-compose.prod.yml down
```

```
[+] down 2/2
✓ Container angular-docker-angular-prod-1 Removed 0.2s
✓ Network angular-docker_default Removed 0.2s
kisna@MacBook-Air-2 angular-docker %
```

Figure 14. Stopping development and production services

Explanation: The development container is removed first, but the shared project network remains while the production container is attached. After the production Compose configuration is stopped, the remaining container and network are removed.


```

Kisna@MacBook-Air-2 angular-docker % docker compose -f docker-compose.prod.yml up --build
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 29
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 30
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 31
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 32
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 33
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 34
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 35
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 36
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 37
angular-prod-1 2026/07/22 07:26:03 [notice] #1: start worker process 38
angular-prod-1 192.168.65.1 -- [22/Jul/2026:07:26:35 +0000] "GET / HTTP/1.1" 200 23141 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"
angular-prod-1 192.168.65.1 -- [22/Jul/2026:07:26:35 +0000] "GET /main-QKPY7UE.js HTTP/1.1" 200 253208 "http://localhost/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"
angular-prod-1 192.168.65.1 -- [22/Jul/2026:07:26:35 +0000] "GET /styles-N3GZVBP.css HTTP/1.1" 200 4544 "http://localhost/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"
angular-prod-1 192.168.65.1 -- [22/Jul/2026:07:26:35 +0000] "GET /favicon.ico HTTP/1.1" 200 15086 "http://localhost/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 3 (SIGQUIT) received, shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 29#29: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 32#32: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 35#35: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 36#36: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 33#33: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 38#38: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 35#35: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 36#36: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 38#38: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 34#34: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 33#33: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 31#31: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 29#29: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 33#33: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 38#38: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 31#31: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 34#34: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 30#30: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 29#29: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 32#32: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 37#37: gracefully shutting down
angular-prod-1 2026/07/22 07:42:30 [notice] 38#38: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 38#38: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 37#37: exiting
angular-prod-1 2026/07/22 07:42:30 [notice] 31#31: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 35#35: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 32#32: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 34#34: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 37#37: exit
angular-prod-1 2026/07/22 07:42:30 [notice] 36#36: exit
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 17 (SIGCHLD) received from 37
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 31 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 37 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 29 (SIGIO) received
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 17 (SIGCHLD) received from 38
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 38 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 29 (SIGIO) received
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 17 (SIGCHLD) received from 35
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 29 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 33 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 35 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 29 (SIGIO) received
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 17 (SIGCHLD) received from 34
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 32 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 34 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 29 (SIGIO) received
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 17 (SIGCHLD) received from 32
angular-prod-1 2026/07/22 07:42:30 [notice] #1: signal 17 (SIGCHLD) received from 36
angular-prod-1 2026/07/22 07:42:30 [notice] #1: worker process 36 exited with code 0
angular-prod-1 2026/07/22 07:42:30 [notice] #1: exit
angular-prod-1 exited with code 0

```

Figure 15. Graceful Nginx shutdown logs

Explanation: After the stop command, Nginx receives termination signals, gracefully shuts down its worker processes, and exits with code 0. This is normal container lifecycle behavior.

```

Kisna@MacBook-Air-2 angular-docker % docker compose up --build
angular-app-1 | styles.css | styles | 6.03 kB |
angular-app-1 | | | |
angular-app-1 | | Initial total | 53.89 kB |
angular-app-1 | | | |
angular-app-1 | Server bundles | | | |
angular-app-1 | Initial chunk files | Names | Raw size |
angular-app-1 | main.server.mjs | main.server | 49.13 kB |
angular-app-1 | server.mjs | server | 1.84 kB |
angular-app-1 | polyfills.server.mjs | polyfills.server | 268 bytes |
angular-app-1 | | | |
angular-app-1 | Application bundle generation complete. [0.897 seconds] - 2026-07-22T07:30:18.141Z
angular-app-1 | | | |
angular-app-1 | Watch mode enabled. Watching for file changes...
angular-app-1 | NOTE: Raw file sizes do not reflect development server per-request transformations.
angular-app-1 | → Local: http://localhost:4200/
angular-app-1 | → Network: http://172.23.0.3:4200/
angular-app-1 | npm error path /app
angular-app-1 | npm error command failed
angular-app-1 | npm error signal SIGTERM
angular-app-1 | npm error command sh -c ng serve --host=0.0.0.0 --port=4200 --allowed-hosts=all
angular-app-1 | npm error A complete log of this run can be found in: /root/.npm/_logs/2026-07-22T07_30_16_949Z-debug-0.log
angular-app-1 | exited with code 1
Kisna@MacBook-Air-2 angular-docker %

```

Figure 16. Angular development process terminated during shutdown

Explanation: The Angular development process reports SIGTERM and exits after the container is stopped. The message represents an intentional termination signal rather than an Angular compilation failure.

Docker Commands Explained

The project uses the following commands in sequence. Each command is shown with its practical purpose and expected behavior.

ng new angular-docker-app

Creates a new Angular workspace and application, generates the source structure, and installs the initial npm dependencies.

docker compose up --build

Reads docker-compose.yml, rebuilds the development image when required, creates the service container and network, starts Angular, and attaches the terminal to the service logs.

docker compose down

Stops and removes the development service containers and removes the Compose network when no other containers are still using it.

docker compose -f docker-compose.prod.yml up --build

Uses the production Compose file, performs the multi-stage Docker build, creates the Nginx-based service, publishes port 80, starts the container, and attaches to logs.

docker compose -f docker-compose.prod.yml down

Stops and removes the production service and its Compose-managed resources.

docker ps

Lists running containers with container IDs, images, commands, uptime, published ports, and names.

docker images

Lists locally available images, image identifiers, disk usage, content size, and whether an image is currently in use.

docker compose ps

Shows the status of services that belong to the current Compose project, including service name, state, and published ports.

Ctrl + C

Sends an interrupt to the foreground Compose process. Docker forwards a termination signal to the container process so it can shut down gracefully.

docker compose up --build --remove-orphans

Optional cleanup form that starts the selected Compose services and removes containers created by older service definitions that are no longer present.

Conclusion

The project successfully demonstrates a complete Angular containerization workflow. Development uses Node.js, Angular's live development server, source mounting, and port 4200. Production uses a multi-stage build to keep compilation dependencies out of the runtime image and serves the final static application through Nginx on port 80. Docker Compose makes both workflows repeatable and provides consistent lifecycle management. The collected evidence confirms successful builds, correct networking, browser accessibility, image creation, service status, and controlled shutdown.